Automatic Classification of RGB Colors

How the Inside the Network simulator labels colors as light or dark

1. The problem and the “Auto-label 10 colors” button

In the *RGB colors: light or dark* problem, each example is a color, described by its red, green, and blue channels and paired with a label: $y = 0$ (dark) or $y = 1$ (light). You can label the colors yourself, in which case the dataset reflects your own judgment. The **Auto-label 10 colors** button, by contrast, generates 10 colors and labels them according to a fixed rule. The rule is based on sRGB relative luminance (sRGB is the standard color space that defines how the three channel values map to actual colors) and on a threshold derived from the contrast-ratio formula in the Web Content Accessibility Guidelines (WCAG), the W3C’s guidelines for web accessibility. The resulting dataset thus follows a known rule, which the network has to learn to approximate from the examples.

Here, “light” and “dark” are a convention: the labels are based on contrast against black and against white, and may not match how an observer would judge the color by eye.

The components R, G, B of each generated color are drawn independently and uniformly from the interval $[0,1)$ by a reproducible pseudorandom generator, so the same seed always produces the same colors. The integer values shown in the interface, $\mathrm{RGB}(r, g, b)$ with $r, g, b \in \{0,1, \dots ,255\}$, are $255R$, $255G$, and $255B$ rounded to the nearest integer; both the labeling rule and the network work with the unrounded values.

The rule has three steps: linearizing the channels, computing the luminance, and comparing it with a threshold.

2. Step 1: linearizing the channels

sRGB values are not proportional to the intensity of the light emitted by the screen. They are encoded nonlinearly (so-called gamma encoding), so that the available levels are spread more evenly with respect to how we perceive brightness. Before the channels are combined, each channel $C \in \{R, G, B\}$ is therefore converted back to a linear scale, proportional to light intensity, using the inverse sRGB transfer function:

$$C_{\mathrm{lin}} = f(C) = \begin{cases} \dfrac{C}{12.92} & \text{if } C \le 0.04045, \\ \left(\dfrac{C + 0.055}{1.055}\right)^{2.4} & \text{if } C > 0.04045. \end{cases}$$

This gives $R_{\mathrm{lin}} = f(R)$, $G_{\mathrm{lin}} = f(G)$, and $B_{\mathrm{lin}} = f(B)$, which still lie between 0 and 1. A few remarks:

- f is increasing, with $f(0) = 0$ and $f(1) = 1$.
- Near zero, f is linear; elsewhere it is a power function with exponent 2.4, shifted and rescaled.
- Overall, $f(C)$ is close to $C^{2.2}$. For example, the mid-gray $\mathrm{RGB}(128,128,128)$, that is $C = 128/255 \approx 0.502$, gives $f(C) \approx 0.216$: about one fifth of the intensity of white, not half.

Note. At the point where the two branches meet, $C = 0.04045$, both are approximately 0.0031308 and they differ by less than $3 \cdot 10^{-9}$: strictly speaking, the function is not continuous, but for all practical purposes it is. Before May 2021, WCAG gave this threshold as 0.03928 rather than 0.04045; the difference has no practical effect. The simulator uses 0.04045, the value specified in the sRGB standard (IEC 61966-2-1).

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

3. Step 2: relative luminance

The three linearized channels are combined into the relative luminance

$$Y = 0.2126\,R_{\text{lin}} + 0.7152\,G_{\text{lin}} + 0.0722\,B_{\text{lin}}.$$

The coefficients depend on the sRGB primaries and on how sensitive the eye is to different wavelengths. They add up to 1, so $Y \in [0,1]$, with $Y = 0$ for black and $Y = 1$ for white. Green carries about ten times as much weight as blue, because the eye is far more sensitive to green light than to blue light.

One immediate consequence is that pure red, green, and blue all have the same channel average, $1/3$, yet very different luminances (0.2126, 0.7152, and 0.0722, respectively).

4. Step 3: the contrast threshold

For two colors with luminances $L_1 \geq L_2$, WCAG defines the contrast ratio

$$\text{CR}(L_1, L_2) = \frac{L_1 + 0.05}{L_2 + 0.05}.$$

The 0.05 term accounts for ambient light reflected off the screen (*viewing flare*) and keeps the ratio from growing without bound for very dark colors. The ratio ranges from 1, for two colors with the same relative luminance, to 21, for black on white.

For a color with luminance Y, the contrast ratios against black ($L = 0$) and against white ($L = 1$) are

$$\text{CR}_{\text{black}}(Y) = \frac{Y + 0.05}{0.05}, \qquad \text{CR}_{\text{white}}(Y) = \frac{1.05}{Y + 0.05}.$$

The first increases with Y and the second decreases. The simulator labels a color “light” when, according to the WCAG formula, its contrast with black is at least as high as its contrast with white. A similar criterion decides whether the text drawn on top of the color swatches is black or white. The threshold is found by setting the two contrast ratios equal:

$$\frac{Y + 0.05}{0.05} = \frac{1.05}{Y + 0.05} \quad \Leftrightarrow \quad (Y + 0.05)^2 = 0.05 \cdot 1.05 = 0.0525,$$

and taking the positive root gives

$$Y^* = \sqrt{0.0525} - 0.05 \approx 0.179129.$$

In other words, $Y^* + 0.05$ is the geometric mean of 0.05 and 1.05. At the threshold, both contrast ratios are equal to $\sqrt{0.0525}/0.05 = \sqrt{21} \approx 4.58$. The threshold is not $1/2$ because contrast is a ratio: the balance point between black and white lies on a multiplicative scale, not an additive one.

5. The complete rule

Putting the three steps together, the label automatically assigned to the color (R, G, B) is

$$y = \begin{cases} 1 \ \text{(light)} & \text{if } Y(R,G,B) \geq Y^*, \\ 0 \ \text{(dark)} & \text{if } Y(R,G,B) < Y^*, \end{cases} \qquad \text{where } Y(R,G,B) = 0.2126\,f(R) + 0.7152\,f(G) + 0.0722\,f(B).$$

By convention, a color with $Y = Y^*$ exactly is labeled light.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

Grays. If $R = G = B = C$, then $Y = f(C)$, because the coefficients add up to 1. A gray is therefore light if and only if

$$C \geq f^{-1}(Y^*) = 1.055\,(Y^*)^{1/2.4} - 0.055 \approx 0.4603,$$

that is, roughly, when $255\,C \geq 117.4$. For grays defined by exact integer RGB components, those up to $\mathrm{RGB}(117,117,117)$ are therefore classified as dark, and those from $\mathrm{RGB}(118,118,118)$ upward as light.

For generated colors, however, the integer values displayed are rounded, so very close to the threshold a color reconstructed from those integers may receive a different label. For example, a gray with $C = 117.45/255$ has $Y \approx 0.1794 > Y^*$ and is therefore light, yet it is displayed as $\mathrm{RGB}(117,117,117)$, and that exact color is dark.

Class proportions. With channels drawn independently and uniformly, the probability that a color is light is about 64.4%. Automatic generation therefore favors the light class and does not guarantee a balanced dataset. When evaluating the network, it is worth also checking the balanced accuracy, which the simulator reports alongside the overall percentage of correctly classified examples.

6. Some examples

In the table, R, G, and B are the integer values shown divided by 255.

Color	$\mathrm{RGB}(r,g,b)$	$(R_{\mathrm{lin}}, G_{\mathrm{lin}}, B_{\mathrm{lin}})$	Y	Class
red	(255,0,0)	(1,0,0)	0.2126	light
green	(0,255,0)	(0,1,0)	0.7152	light
blue	(0,0,255)	(0,0,1)	0.0722	dark
yellow	(255,255,0)	(1,1,0)	0.9278	light
magenta	(255,0,255)	(1,0,1)	0.2848	light
gray	(128,128,128)	(0.2159,0.2159,0.2159)	0.2159	light
gray	(117,117,117)	(0.1779,0.1779,0.1779)	0.1779	dark
purple	(128,0,128)	(0.2159,0,0.2159)	0.0615	dark
olive	(128,128,0)	(0.2159,0.2159,0)	0.2003	light
steel blue	(70,130,180)	(0.0612,0.2232,0.4564)	0.2056	light

Purple and olive have the same channel average, about 0.335, yet they fall into opposite classes: the green channel makes the difference. The gray $\mathrm{RGB}(117,117,117)$ lies just below the threshold $Y^* \approx 0.1791$.

7. Why a hidden layer can help

The network receives the normalized sRGB values R, G, and B, rather than the linearized channels. In input space, the cube $[0,1]^3$, the region of light colors is bounded by the surface

$$0.2126\,f(R) + 0.7152\,f(G) + 0.0722\,f(B) = Y^*.$$

For the sRGB linearization function, this surface is curved: it is not a plane.

A network with no hidden layers computes $\hat{y} = \sigma(w_1 R + w_2 G + w_3 B + b)$ and, with a threshold of $1/2$, separates the classes with the plane $w_1 R + w_2 G + w_3 B + b = 0$, so it can only approximate the curved surface. A numerical search for the plane that best classifies the colors, evaluated on the uniform distribution, gave an accuracy of about 94%, with the errors concentrated near the decision boundary. This is an empirical finding, not a proof of optimality, and it is not a bound that applies to every finite sample: a small set of examples may well be perfectly

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.

separable by a plane. Note also that training in the simulator does not look for this plane directly: it minimizes the squared error.

With enough neurons and a nonlinear activation function, a hidden layer allows the network to approximate the curved decision boundary more closely. The simulator offers three such activation functions: hyperbolic tangent, sigmoid, and ReLU. This extra capacity can improve how well new colors are classified, but the result also depends on the available examples and on training, so it should be checked on the validation set.

If, instead, the network were given the linearized channels directly, the decision boundary would be the plane

$$0.2126\,R_{\text{lin}} + 0.7152\,G_{\text{lin}} + 0.0722\,B_{\text{lin}} = Y^{*}.$$

In these coordinates, even a network with no hidden layers could represent the separation exactly. A single output neuron with weights $(0.2126, 0.7152, 0.0722)$ and bias $-Y^{*}$ is sufficient: its pre-activation $z = Y - Y^{*}$ is non-negative exactly when the color is labeled light. Multiplying all the weights and the bias by the same positive factor preserves the sign of z, and therefore the classification. So whether a nonlinear boundary is needed depends, in part, on how the input is represented.

References

- W3C, Understanding Success Criterion 1.4.3: Contrast (Minimum), WCAG 2.2: definitions of relative luminance and contrast ratio, with notes on the former 0.03928 threshold and on the 0.05 term.
- IEC 61966-2-1:1999, *Multimedia systems and equipment — Colour measurement and management — Part 2-1: Default RGB colour space — sRGB*.
- Simulator source code, file `en/index.html`: functions `linearSrgbChannel`, `relativeSrgbLuminance`, and `classifyRgbColorsAutomatically`.

© Francesco Operetto, 2026 · Licensed under CC BY-SA 4.0
Draft version: may contain errors or inaccuracies. Comments and corrections are welcome.
Last updated: October 19, 2026.
This text was written and revised with the assistance of artificial intelligence tools.